

Minimal System Diagram Set (Product • Architecture • QA • DevOps • SRE • Design • Engineering)

Diagram	Primary Purpose	Must Include (Minimum Fields)	Typical Questions This Diagram Answers	Common Mistakes to Avoid	Most Useful For (Teams)
System Context Diagram (C4-L1)	Define boundary. Show who interacts and what connects.	System name, external actors, external systems, high-level flows	Who are the users? What's in/out of scope? What upstream/downstream systems exist?	Mixing internals (services/DBs) into context. Missing ownership boundaries.	Product, Design, Architecture, QA, SRE
Container Diagram (C4-L2)	Explain major internal components and responsibilities.	UI apps, gateway/BFF, core services, datastores, queue/broker, integrations	What are the major components? Who owns what? Where does data live?	Over-granularity (too many boxes). No clear responsibility labels.	Architecture, Engineering, QA, SRE, DevOps
E2E Sequence Diagram (Happy Path)	Make execution flow explicit across services.	Trigger, hops, reads/writes, external calls, response path, timeouts	What happens when user clicks X? Where can latency happen? What breaks if Y is slow?	Only showing services but not data writes. No timeout/retry semantics.	QA, Engineering, SRE, Product
Domain Model Diagram (Bounded Context)	Shared language for business logic. Prevent	Context boundaries, key entities, relationships,	What is an Order vs Checkout? Who owns Pricing? What	Turning it into DB schema. Unbounded "god domain".	Product, Engineering, Architecture, QA

Context + Entities)	mismatch in assumptions.	source-of-truth	is canonical truth?		
State Machine Diagram (Critical Object)	Define lifecycle and allowable transitions.	States, transitions, invalid transitions, triggers/actions, terminal states	What transitions are allowed? What states cause stuck orders? What can be retried?	Skipping edge/failure states. Not marking terminal states clearly.	QA, SRE, Product, Engineering
Data Flow Diagram (PII + Trust Boundaries)	Security/compliance truth. Data movement clarity.	PII tags, trust boundaries, encryption in-transit/at-rest, audit logging, retention	Where is PII stored? Who can access it? Where do we encrypt/mask?	Hand-wavy “data is secure”. Not labeling PII. No retention/deletion flow.	SRE, DevOps, QA, Architecture, Product
Deployment Architecture Diagram	Runtime reality of the system.	Environments, regions/AZs, ingress/LB, compute placement, DB/cache/broker placement, network controls	Where does it run? HA strategy? Blast radius? How do releases roll out?	Showing logical view only (same as container diagram). No AZ/region separation.	DevOps, SRE, Engineering, QA
Observability + Operations Diagram	Make it operable, measurable, supportable.	SLIs/SLOs, metrics/logs/traces flow, alerts, dashboards, runbooks, escalation	How do we detect issues? What triggers a page? What’s our SLO and error budget?	Alerts without SLOs. No runbooks. No ownership routing.	SRE, DevOps, Engineering